[3 주차] 코랩 표준템플릿

강사용 코드 해설본 — 코딩을 직접 하지 않아도 학생 코드를 읽고 설명할 수 있도록

1. 이 문서의 목적

이 문서는 "3 주차_코랩_표준템플릿.ipynb"의 구조와, 그 템플릿을 실제로 채워 넣은 예시(박스오피스 월간 집계 실습)의 코드를 한 줄씩 풀어 설명합니다. 목표는 김사부님이 직접 코드를 짜실 수 있게 되는 것이 아니라, 학생이 짠 코드나 실습가이드에 적힌 코드를 보았을 때 "이 줄이 대략 무엇을 하는 코드인지"를 바로 설명할 수 있는 수준의 문해력을 갖추는 것입니다.

모든 주차 노트북(박스오피스, 미세먼지, 팩트체크, RAG 데모 등)이 이 템플릿과 같은 3 단계 구조를 그대로 재사용하기 때문에, 이 한 편을 이해해두면 다른 주차 코드를 볼 때도 같은 틀로 해석할 수 있습니다.

2. 왜 이런 3 단계 구조인가?

표준템플릿은 매 실습을 아래 3 단계로 강제합니다. 이렇게 나누는 이유는 학생들이 에러가 났을 때 "어느 단계에서 막혔는지"를 스스로 진단하게 만들기 위해서입니다.

- 셀 1. 준비 — 한글 폰트 등, 그래프를 그리기 전에 한 번만 해두면 되는 환경 설정
- 셀 2. 설정 + 수집 — 인증키를 입력하고, 외부 사이트(API)에서 데이터를 가져와 표 형태로 정리
- 셀 3. 결과 확인 — 정리된 표를 파일로 저장하고, 그래프로 그리고, 핵심 숫자를 요약 출력

💡 **강의 팁** 학생이 "에러 났어요"라고 하면 제일 먼저 물어볼 질문은 "몇 번 셀에서 멈췄어요?"입니다. 1 번 셀이면 폰트/설치 문제, 2 번 셀이면 인증키나 API 문제, 3 번 셀이면 앞 단계에서 만든 표(DataFrame)의 이름이나 형태가 안 맞는 문제일 가능성이 큽니다.

3. 셀 1 — 한글 폰트 설치 (실제 코드)

코랩(Google Colab)은 내부적으로 리눅스 컴퓨터를 빌려 쓰는 서비스인데, 이 리눅스에는 한글 폰트가 기본으로 깔려있지 않습니다. 그래서 매번 실습 맨 앞에서 폰트를 설치해주는 코드가 반복됩니다.

```
!apt-get -qq -y install fonts-nanum > /dev/null

import matplotlib.pyplot as plt
import matplotlib.font_manager as fm

font_path = "/usr/share/fonts/truetype/nanum/NanumGothic.ttf"
fm.fontManager.addfont(font_path)
plt.rc("font", family="NanumGothic")
plt.rcParams["axes.unicode_minus"] = False  # 마이너스(-) 기호 깨짐 방지

print("폰트 준비 완료...")
```

코드	무슨 뜻인가
`!apt-get -qq -y install fonts-nanum`	느낌표(!)로 시작하면 "이건 파이썬 코드가 아니라 리눅스

코드	무슨 뜻인가
	컴퓨터에 내리는 설치 명령이다"라는 뜻입니다. "나눔고딕(fonts-nanum)이라는 폰트를 조용히(-qq), 묻지 말고 자동으로(-y) 설치해라"는 명령입니다.
`> /dev/null`	설치 과정에서 나오는 지저분한 로그를 화면에 안 띄우고 그냥 버리라는 뜻입니다. (휴지통에 버린다고 보시면 됩니다)
`import matplotlib.pyplot as plt`	"matplotlib"이라는 그래프 그리기 도구상자를 가져와서, 앞으로는 짧게 "plt"라는 별명으로 부르겠다는 선언입니다.
`import matplotlib.font_manager as fm`	그 도구상자 안에서 "어떤 폰트를 쓸지 관리하는 부분"만 따로 가져와서 "fm"이라는 별명을 붙입니다.
`font_path = "...NanumGothic.ttf"`	방금 설치한 폰트 파일이 컴퓨터 안 어디에 저장되어 있는지, 그 "주소(경로)"를 font_path 라는 이름의 상자에 담아둡니다. 이렇게 값을 이름에 담아두는 것을 "변수"라고 부릅니다.
`fm.fontManager.addfont(font_path)`	"이 주소에 있는 폰트 파일을, 그래프 그릴 때 쓸 수 있는 폰트 목록에 등록해줘"라는 명령입니다.
`plt.rc("font", family="NanumGothic")`	"이제부터 그래프에 글자를 쓸 때는 기본으로 나눔고딕을 써라"는 설정입니다.
`plt.rcParams["axes.unicode_minus"] = False`	한글 폰트를 쓰면 마이너스(-) 기호가 네모(□)로 깨지는 고질적인 버그가 있어서, 이를 막아주는 설정입니다. 없으면 그래프의 음수 눈금이 깨져 보입니다.
`print("...")`	지금까지 잘 됐는지 화면에 메시지를 출력해서 학생이 눈으로 확인하게 해주는 줄입니다.

💡 **강의 팁** 여기 나오는 import, 변수, 함수 호출(괄호를 쓰는 것) 패턴은 이후 모든 코드에서 계속 반복됩니다. 이 셀 하나만 정확히 짚어드려도 "코드를 읽는 기본 문법"의 절반은 설명이 끝납니다.

4. 셀 2 — 설정 + 데이터 수집 (개념 + 실전 예시)

표준템플릿의 셀 2 는 아래 네 단계를 "한 셀 안에" 모두 적으라고 안내만 되어 있고, 실제 코드는 매주 실습마다 다르게 채워집니다. 여기서는 "3 주차_박스오피스_월별_코랩.ipynb"에 실제로 채워진 코드를 예시로 가져와서 설명합니다.

- 설정값 — 인증키, 조회할 날짜/대상 등을 미리 적어두는 부분
- API 호출 / 함수 정의 — 외부 사이트에 "데이터 주세요" 요청을 보내는 부분
- 실제 데이터 수집(반복문) — 같은 요청을 여러 번 반복해서 데이터를 모으는 부분
- DataFrame 으로 정리 — 모은 데이터를 표 형태로 깔끔하게 정리하는 부분

4-1. "API"와 "인증키"란 무엇인가

API(Application Programming Interface)는 쉽게 말해 "기관 홈페이지 뒷문으로 데이터를 요청하는 창구"입니다. 사람이 홈페이지 화면을 눈으로 보는 대신, 프로그램이 정해진 형식으로 "이 날짜 데이터 주세요"라고 요청하면, 그 기관 서버가 표/숫자 형태의 데이터(JSON)로 바로 응답해주는 것입니다.

인증키(API Key)는 "이 요청을 보낸 사람이 정식으로 등록된 사용자다"를 증명하는 비밀번호 같은 문자열입니다. 영화진흥위원회(KOBIS), 기상청, 공공데이터포털 같은 곳에 가입하면 발급받을 수 있고, 이 키 없이는 데이터를 안 줍니다.

4-2. 실전 코드 — KOBIS 박스오피스 월간 집계

영화진흥위원회(KOBIS)는 "이번 달 전체 통계"를 한 번에 주는 API 가 없어서, "하루치 통계"를 31 번 요청해서 직접 더하는 방식으로 만든 코드입니다. 아래는 실제 코드 전체이고, 이어서 핵심 부분만 발췌해 설명합니다.

```
import requests
import csv
import time
from datetime import datetime, timedelta

KOBIS_KEY = "여기에_발급받은_인증키_입력"
url = "http://kobis.or.kr/kobisopenapi/.../searchDailyBoxOfficeList.json"

# 1) 7월 1일부터 31일까지 날짜 목록 만들기
start_date = datetime(2026, 7, 1)
end_date = datetime(2026, 7, 31)
date_list = []
current = start_date
while current <= end_date:
    date_list.append(current.strftime("%Y%m%d"))
    current += timedelta(days=1)
```

코드	무슨 뜻인가
`import requests`	"requests"는 인터넷 저편 서버에 요청을 보내고 응답을 받아오는, 파이썬에서 가장 많이 쓰는 도구입니다. 학생들이 "API 호출"이라고 말하면 대부분 이 도구를 쓴다는 뜻입니다.
`import csv / import time`	csv 는 표 데이터를 파일로 저장/읽기 위한 도구, time 은 "몇 초 쉬었다 가기" 같은 시간 관련 기능을 위한 도구입니다.
`from datetime import datetime, timedelta`	날짜를 다루는 도구입니다. datetime 은 "2026 년 7 월 1 일" 같은 특정 날짜, timedelta 는 "1 일" 같은 시간 간격을 표현합니다.
`KOBIS_KEY = "..."`	발급받은 인증키를 변수(이름표 붙은 상자)에 담아둡니다. 실제 사용 시 학생은 본인이 발급받은 키로 이 따옴표 안을 바꿔 넣어야 합니다.
`url = "http://..."`	데이터를 요청할 KOBIS 서버의 정확한 주소입니다.
`start_date / end_date`	"언제부터 언제까지"를 각각 변수에 담아둡니다.

코드	무슨 뜻인가
`date_list = []`	빈 리스트(순서가 있는 목록 상자)를 하나 만듭니다. 앞으로 여기에 "20260701", "20260702"... 같은 날짜들을 하나씩 채워 넣을 예정입니다.
`while current <= end_date: ...`	"반복문"입니다. current(현재 날짜)가 end_date(끝 날짜)를 넘지 않는 동안 계속 같은 동작(날짜를 목록에 추가하고 하루를 더함)을 반복하라는 뜻입니다. 사람이 31 번 직접 타이핑하는 대신 컴퓨터가 대신 반복해주는 것이 프로그래밍의 핵심 가치입니다.
`date_list.append(...)`	date_list 라는 목록 상자 끝에 항목을 하나 추가합니다.
`current += timedelta(days=1)`	current 날짜에 하루를 더해서 다음 날짜로 넘어갑니다.

```
movie_totals = {}

for i, date_str in enumerate(date_list):
    params = {"key": KOBIS_KEY, "targetDt": date_str}
    response = requests.get(url, params=params)
    data = response.json()
    daily_list = data["boxOfficeResult"]["dailyBoxOfficeList"]

    for movie in daily_list:
        title = movie["movieNm"]
        audience = int(movie["audiCnt"])
        if title in movie_totals:
            movie_totals[title]["audience"] += audience
        else:
            movie_totals[title] = {"audience": audience}

    time.sleep(0.2)
```

코드	무슨 뜻인가
`movie_totals = {}`	빈 "딕셔너리"를 만듭니다. 딕셔너리는 "영화 제목"이라는 이름표에 "누적 관객수"라는 값을 매달아두는 서랍장이라고 생각하시면 됩니다. (리스트가 순서만 있는 목록이라면, 딕셔너리는 이름으로 찾는 서랍장입니다.)
`for i, date_str in enumerate(date_list):`	date_list 에 담긴 날짜를 하나씩 꺼내면서, 그 아래 문장들을 날짜 개수(31 번)만큼 반복하라는 뜻입니다. enumerate 는 "몇 번째인지(i)"도 같이 알려달라는 부가 기능입니다.
`params = {"key": ..., "targetDt": ...}`	서버에 보낼 "질문지"를 만드는 것입니다. "내 인증키는 이거고, 조회하고 싶은 날짜는 이거야"를 정리한 것입니다.
`response = requests.get(url, params=params)`	실제로 서버에 "이 주소로, 이 질문지를 들고 데이터 요청"을 보냅니다. 이 한 줄이 바로 "API 를 호출한다"는 행위입니다.
`data = response.json()`	서버가 돌려준 응답을, 파이썬이 다룰 수 있는 표/딕셔너리 형태(JSON)로 변환합니다.

코드	무슨 뜻인가
`daily_list = data["boxOfficeResult"]["..."]`	응답 데이터 안에서, 실제로 필요한 "그날의 영화 10 개 목록" 부분만 꺼냅니다. (대괄호로 서랍을 하나씩 열어 들어가는 것과 같습니다)
`for movie in daily_list: ...`	그날의 영화 10 개를 하나씩 꺼내면서 아래 로직을 반복합니다. (반복문 안에 반복문이 또 있는 구조 — "이중 반복문"이라 부릅니다)
`title = movie["movieNm"] / audience = int(...)`	영화 하나에서 "제목"과 "관객수"만 꺼내서 변수에 담습니다. int(...)는 문자로 되어있는 숫자를 진짜 숫자로 바꿔주는 것입니다.
`if title in movie_totals: ... else: ...`	"이 영화가 이미 서랍장에 있으면 기존 숫자에 더하고, 처음 나온 영화면 새 서랍을 만들어라"는 조건 분기입니다. 이렇게 31 일치를 다 더하면 "월간 누적"이 완성됩니다.
`time.sleep(0.2)`	요청 하나 보낼 때마다 0.2 초씩 쉬어가는 것입니다. 너무 빠르게 연속 요청하면 상대 서버에 부담을 주거나 차단당할 수 있어서, 예의상 속도를 조절하는 것입니다.

💡 **강의 팁** 여기서 학생들이 가장 헷갈려하는 지점은 "이중 반복문"입니다. "바깥 반복문은 날짜(31 일)를 돌고, 그 안에서 또 영화(10 개)를 도니까 총 310 번 도는 것"이라고 숫자로 풀어서 설명해주시면 이해가 훨씬 빠릅니다.

5. 셀 3 — 결과 확인: 저장 + 시각화 + 요약

박스오피스 예시 코드의 마지막 부분(정렬 → 출력 → CSV 저장)으로 셀 3 의 개념을 설명합니다.

```
top10 = sorted(movie_totals.items(), key=lambda x: x[1]["audience"], reverse=True)[:10]

for rank, (title, stats) in enumerate(top10, start=1):
    print(f"{rank}위 | {title} | 관객수 {stats['audience']:,}명")

csv_filename = "boxoffice_2026_07_monthly.csv"
with open(csv_filename, "w", newline="", encoding="utf-8-sig") as f:
    writer = csv.writer(f)
    writer.writerow(["순위", "영화제목", "누적관객수"])
```

코드	무슨 뜻인가
`sorted(..., key=..., reverse=True)[:10]`	서랍장(딕셔너리)에 있던 영화들을 관객수 기준으로 "많은 순"으로 줄 세운 뒤, 앞에서 10 개만 잘라옵니다. 이게 바로 "TOP 10"을 만드는 코드입니다.
`f"{rank}위 | {title} | 관객수 {stats['audience']:,}명"`	f 로 시작하는 문자열(f-string)은 "중괄호 { } 안의 변수 값을 그 자리에 끼워넣어서 문장을 완성해줘"라는 뜻입니다. 콤마(:,)는 "천 단위마다 쉼표를 넣어줘"라는 숫자 표시 옵션입니다 (예: 1234567 → 1,234,567).
`open(csv_filename, "w", ...,`	새 파일을 "쓰기(w)" 모드로 엽니다. encoding="utf-8-

코드	무슨 뜻인가
`encoding="utf-8-sig")`	sig"는 엑셀에서 한글이 깨지지 않게 해주는 저장 방식으로, 한글 데이터를 다룰 땐 거의 항상 이 옵션을 씁니다.
`writer = csv.writer(f) / writer.writerow([...])`	열어둔 파일에 표의 한 줄(행)씩 적어 넣습니다. writerow 에 넣은 리스트의 각 항목이 엑셀의 한 칸(셀)씩 채워집니다.

템플릿의 "시각화" 단계는 이 예시엔 없지만, 다른 주차(미세먼지, 추석 날씨 등)에서는 matplotlib(plt.plot, plt.bar 등)으로 꺾은선·막대그래프를 그립니다. 원리는 "정리된 표의 특정 열(column)을 x 축/y 축에 넣고 그려라"는 명령을 내리는 것으로, 셀 1 에서 설정해둔 한글 폰트가 여기서 실제로 쓰입니다.

6. 부록 — 진단(디버그) 셀

템플릿 맨 아래 "debug=True" 같은 진단용 코드를 넣으라는 안내가 있습니다. 이는 학생이 "API 가 데이터를 잘 주고 있는지"를 중간에 눈으로 확인해볼 수 있게, print(response.status_code)나 print(data) 같은 줄을 임시로 추가해보는 자리입니다. "켰다 껐다 할 수 있는 손전등"이라고 설명하시면 이해가 쉽습니다.

7. 매주 반복해서 나오는 용어 사전

아래 용어들은 3 주차뿐 아니라 이후 주차(RAG, 파인튜닝 포함) 실습가이드에도 계속 등장합니다. 한 번 정리해두시면 매주 재사용하실 수 있습니다.

용어	쉬운 설명 (학생에게 쓸 수 있는 말)
라이브러리	누군가 미리 만들어둔 "도구 상자". import 로 불러와서 씁니다. (예: matplotlib= 그래프 그리기 상자, requests=인터넷 요청 상자)
변수	값에 이름을 붙여 담아두는 상자. (예: KOBIS_KEY = "abc123")
함수	정해진 동작을 하나로 묶어 이름 붙여둔 것. 괄호()를 붙여서 "실행해줘"라고 호출합니다. (예: print(), sorted())
반복문 (for / while)	같은 동작을 여러 번 자동으로 반복시키는 문법. "31 번 요청 보내기"처럼 사람이 직접 반복할 일을 컴퓨터에게 맡깁니다.
조건문 (if / else)	"~라면 이렇게, 아니면 저렇게" 갈림길을 만드는 문법.
리스트 [ ]	순서가 있는 목록 상자. (예: 날짜 31 개를 순서대로 담은 목록)
딕셔너리 { }	이름(키)으로 값을 찾는 서랍장. (예: "영화제목"이라는 이름표로 "관객수"를 꺼냄)
API	외부 기관 서버에 정해진 형식으로 데이터를 요청하는 창구. "인터넷으로 데이터 주문하기"에 가깝습니다.
인증키 (API Key)	그 API 를 쓸 자격이 있다는 걸 증명하는 비밀 문자열. 발급받아야 사용 가능합니다.
JSON	API 가 돌려주는, 프로그램이 읽기 좋은 표/서랍장 형태의 데이터 포맷.
DataFrame	표(엑셀 시트 같은 것)를 파이썬 안에서 다루기 위한 판다스(pandas) 라이브러리의 핵심 도구.

용어	쉬운 설명 (학생에게 쓸 수 있는 말)
CSV	표 데이터를 저장하는 가장 기본적인 파일 형식. 엑셀로도 바로 열립니다.
주석 (#)	코드가 아니라 사람이 읽으라고 남긴 설명 메모. # 뒤에 적힌 글은 컴퓨터가 무시합니다.

8. 학생에게 설명할 때 쓸 수 있는 비유 모음

- 라이브러리 = "방송 장비 대여소" — 카메라를 직접 만들지 않고 빌려 쓰듯, 코드도 남이 만든 기능을 빌려(import) 씁니다.
- API 호출 = "방송사 자료 요청 공문" — 정해진 양식(인증키+날짜)을 갖춰 보내야 자료를 내어줍니다.
- 반복문 = "같은 촬영을 앵글만 바꿔 31 번 찍는 것" — 사람이 31 번 반복할 일을, 컴퓨터에게 "이 동작을 31 번 해"라고 한 번만 지시하는 것입니다.
- 딕셔너리 = "이름표 붙은 서랍장" — 순서가 아니라 이름으로 바로 찾아 꺼냅니다.
- 변수 = "소품에 붙인 이름표" — 나중에 그 이름만 부르면 어떤 값이었는지 다시 안 찾아도 됩니다.

※ 이 문서는 3 주차_코랩_표준템플릿.ipynb 와 3 주차_박스오피스_월별_코랩.ipynb 의 실제 코드를 근거로 작성되었습니다. 다른 주차(팩트체크, 미세먼지, RAG 데모 등)도 같은 3 단계 구조를 재사용하므로, 필요하시면 이어서 해설본을 만들어 드릴 수 있습니다.